

Software Risk Tracker



BY:

Emily, Alli, Hannah, Nick, Reddy, Brandon, Logan,
Josh, and Shelby

Project Requirements

Understanding the Requirements

- Team relied on the use of SRS to ensure all the requirements were met
- Once we finished the SRS everyone was able to understand the specific requirements of each team
- Going from that we were able to break down the requirements into sprints and use that as a guide on what to do for each week on the project.
- We did have small problems of adding stuff that was not a requirements but as we neared the end we adjusted for time and removed any unfinished non- required features.

How to Improve for the Requirements

- One of the lessons we learned was the importance of testing before merging, additionally making sure your branch is up to date.
- We also learned the importance of the testing against all requirements
- Also, we learned to develop our code on the CS server to prevent the problems from occurring in the first place
 - Ensured good code practices
- Better communication, coordination, and hierarchy with our teammates.
- Follow the sprint outline structure more closely and appointed a Scrum Master
- A better base understanding of git

Project Design

Understanding of the Design Process

- The team had a shared high-level understanding of the design early-on: The Software Design Document (sdl/sdd.tex) lays out client-server Witty/Wt architecture, model-view decomposition, major subsystems (UI, application layer, auth, risk management, persistence), and class/package diagrams.
- Similarly, code layout matched the mental model: src/model vs src/view, plus named managers (UserManager, RiskManager, ProjectManager, DatabaseManager), mirrors the subsystem breakdown in the design, which is easy for anyone joining the development process to find the right layer.
- Session architecture was implemented clearly: riskTrackerApp wires DatabaseManager → Authenticator / UserManager / RiskManager / ProjectManager in one place, which matches “single app entry, injected dependencies” intent.

Understanding of the Design Process

- Requirements were followed and abided by well throughout the whole process: sdl/RTVM.md ties requirements IDs to specific implementation files and marks tests passed where filled, evidence the team connected design to code and verification, not just diagrams.
- There was a common vocabulary and structure for engineers, PMs, and testers as this project is giving us a “taste” of working in industry.

Integration

- Having multiple teams working in different areas increase productivity but comes with its own problems.
- Integrating different parts together showed the importance of naming conventions.
- Once all parts are connected together new unforeseen problems commonly arise.
- Fixing these problems requires effective communication between teams.
- Cross training with the separate code helps finish up the merging of the project to a final product.

Possible Improvements to the Design Process

- Tighten vocabulary and naming conventions: Align terms once (username vs email, field names, UI labels) and reflect them in schema, APIs, and docs so reviews focus on behavior, not naming drift.
- Making sure the visual design matches the requirements
- Spreading the workload over multiple teams and people instead of just a couple members of the team
- Outline specific tasks for specific people on the design to help evenly distribute work to ensure nobody does the same task

Traceability Matrix

AUM, SoD	SECU-4	Record logins, create, delete, edit and CSV exports	dbTest.md - /tests/dbTest	PASS
AUM	SECU-5	Reset a users password via admin	systemVerification.md	PASS
Style	QUAL-1	Follow Mozilla C++ coding style	styleTest.md	PASS
Style	QUAL-2	Follow SEI secure coding style (Cppcheck)	qual2Test.md	PASS
UI	QUAL-3	Meet WCAG 2.1 accessibility standards	Qual3.md	PASS
UI, API	FUNC-1	Create project by defining a project name	userTest.md - /tests/userTest AccessModelT	PASS
AUM	FUNC-2	Auth required before accessing project data	userTest.md - /tests/userTest	PASS
UI, RM	FUNC-3	Input risk data (ID, Desc, Likelihood, Impact)	AccessModelT - AccessModelT.md	PASS
UI, API	FUNC-4	Define matrix dimensions during project creation	AccessModelT - AccessModelT.md	PASS
SoD	FUNC-5	Store all data and provide CSV export feature	systemVerification.md	PASS
UI, API	FUNC-6	Dynamic color-coded matrix (Likelihood × Impact)	systemVerification.md	PASS
AUM	FUNC-7	Regular users can access and edit projects	dbTest.md - /tests/dbTest, /tests/riskManagerTest, dbm.md - /tests/dbmTest	PASS
AUM	FUNC-8	Admin users can manage all users and projects	dbTest.md - /tests/dbTest, userTest.md - /tests/userTest, dbm.md - /tests/dbmTest	PASS
UI	FUNC-9	Comply with WCAG accessibility guidelines	Func9.md	PASS

User Interface live tests

2. Performance Metrics Summary

Req ID	Requirement Description	Status	Comments/Methods
PERF-1	20 Concurrent Users Support < 2.0s	PASS	Allie opened 10 tabs and logged in, and performed actions in each tab. Hannah did the same.
PERF-2	Risk create edit delete risks response time < 2.0s	PASS	Created, Deleted, and Edited Users
PERF-3	Save risk input response time < 1.0s	PASS	Saved Project
PERF-4	Initial Page Load < 5.0s	PASS	Connect to the application.
PERF-5	Risk matrix generation time < 4.0s	PASS	Load up the Risk Matrix
PERF-6	Matrix update after risk change < 2.0s	PASS	Made a risk and logged out and logged back in to ensure it stayed stored.
PERF-7	CSV export completion time < 2.0s	PASS	Exported CSV
SAFE-1	Require user to confirm any deletion action	PASS	Created and deleted projects and users
SAFE-2	Auto-logout session after 15m of inactivity	PASS	Waited 15 min to be timed out.
SEC-2	User data securely stored against data loss	PASS	Created Users and Projects, Logged out and Logged in, the data was still there.
SEC-5	Reset a user's password via admin	PASS	Login as admin and reset user's password
FUNC-5	Store all data and provide CSV export feature	PASS	Data is saved in the download
FUNC-6	Dynamic color-coded matrix (Likelihood × Impact)	PASS	Color-Coded matrix is displayed with Likelihood and Impact

3. Verification Sign-off

Role	Name	Date	Status
Lead Tester	Hannah Lanier	2026-05-01	VERIFIED
Peer Reviewer	Alli Ornelas	2026-05-01	VERIFIED

QUAL-1

Follow Mozilla C++ coding style (verified by styleTest.md)

Coding Standards Verification: Mozilla C++ Style

1. Overview

Requirement QUAL-1 requires the source code to adhere to the Mozilla C++ Style Guidelines. This ensures maintainability, readability, and consistency across the project modules.

2. Style Audit Criteria

The following Mozilla-specific conventions were audited across the `/src` directory:

Convention	Mozilla Standard	Implementation in myRisk	Status
Indentation	2 Spaces	Consistent 2-space indentation applied.	PASS
Class Naming	PascalCase (e.g. UserModel)	All models and views use PascalCase.	PASS
Variable Naming	camelCase (e.g. userName)	Member variables follow camelCase.	PASS
Brace Style	Attached (Same line)	Braces start on the same line as declarations.	PASS
File Headers	Include Guards	All .h files use <code>#ifndef</code> or <code>#pragma once</code> .	PASS

3. Code Snippet Verification

Example from `riskItemModel.h` showing compliance:

```
class RiskItem {
public:
    void setProbability(int p) {
        probability = p;
    }
private:
    int probability;
};
```

QUAL-2

Follow SEI secure coding style (Cppcheck) (verified by qual2Test.md)

Static Code Analysis: Cppcheck Security & Style

1. Overview

Requirement QUAL-2 specifies that the system must undergo static analysis to ensure compliance with secure coding standards and to identify potential memory leaks, undefined behaviors, or performance bottlenecks. This test was performed using Cppcheck on the entire /src directory.

2. Analysis Results Summary

Metric	Result
Files Checked	24
Critical Errors	0
Security Flaws	0
Memory Leaks	0
Style Warnings	12
Performance Suggestions	6

QUAL-3

Meet WCAG 2.1 accessibility standards (verified by Qual3.md)

Requirement Verification: QUAL-3 (Accessibility Standards)

Req. ID	Description	Source	Verification Method	Status
QUAL-3	Meet WCAG 2.1 Standards	Capstone Specs	Automated Audit & Reflow Testing	Pass

Verification Details

- **Standards Framework:** Verified against WCAG 2.1 Level AA Success Criteria.
- **Audit Results (Lighthouse):**
 - **Perceivable (Success):** All non-text content provides text alternatives and maintains high contrast ratios for readability.
 - **Understandable (Success):** Web pages appear and operate in predictable ways. Form input assistance (labels/buddy links) verified.
 - **Robust (Success):** Content is compatible with current and future user agents, including assistive technologies.
- **Reflow & Zoom Verification:**
 - **Test Procedure:** Application scaled to 200% zoom.
 - **Result:** No loss of information or functionality; Wt layout responded correctly without horizontal scrolling.
- **Landmark Verification:**
 - **Mitigation:** Applied `setTagName("main")` to the root container to satisfy ARIA landmark requirements for screen reader navigation.

Demo

Recaps and Questions

1. PULL REQUESTS

Small, focused changes are easier to review than large ones.

2. CODE REVIEW

Reading and testing teammates' code is just as important as writing your own.

3. TEAM DYNAMICS

Splitting work across model, view, and test teams requires constant communication.

4. GIT WORKFLOW

Resolving merge conflicts gets easier with practice, but a clear git workflow from day one helps everyone

KEY TAKEAWAY

Communication is KEY to a smooth working process with many moving parts.